# Introduction to Computer Architecture and Systems

## Chapter 8
## C Programing Language

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

# Get access to Linux

# How can I try Linux?

**You can try Linux with hardware and software.**

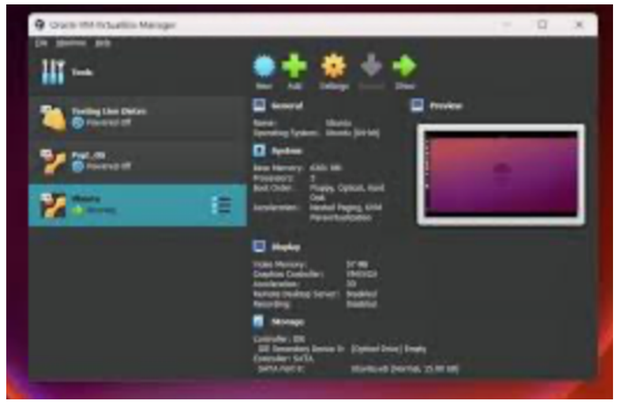
- Hardwares: Raspberry Pi, nVidia Jetson

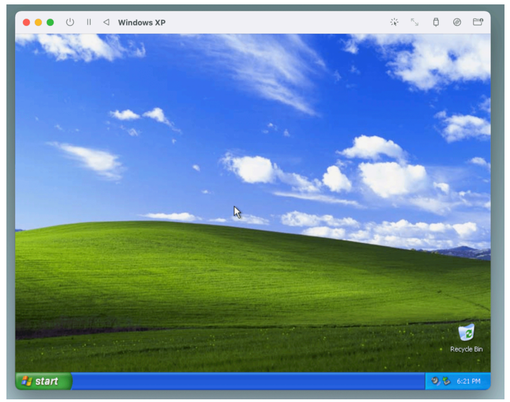





- Softwares: Virtual Machines like VirtualBox(Oracle), UTM(for Apple w ARM)

VirtualBox



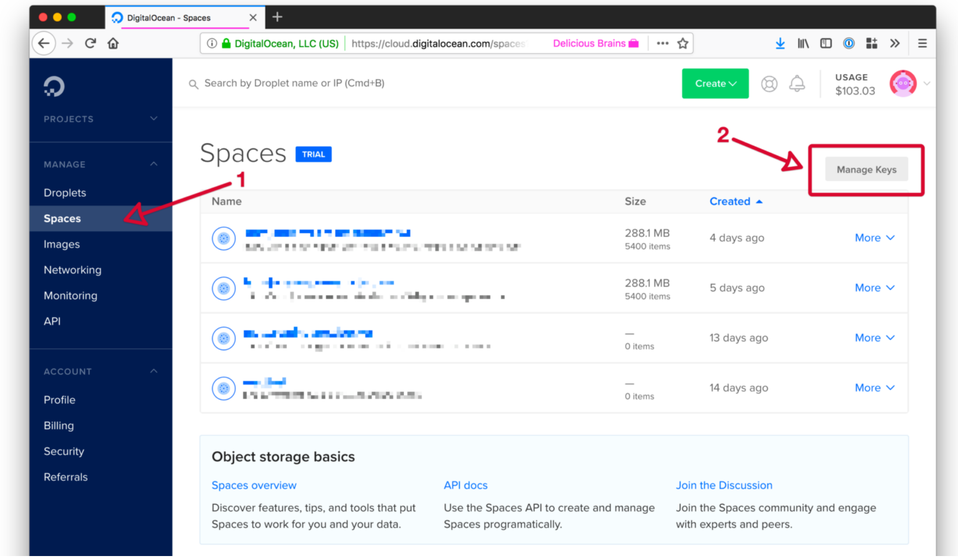


# How can I try Linux?

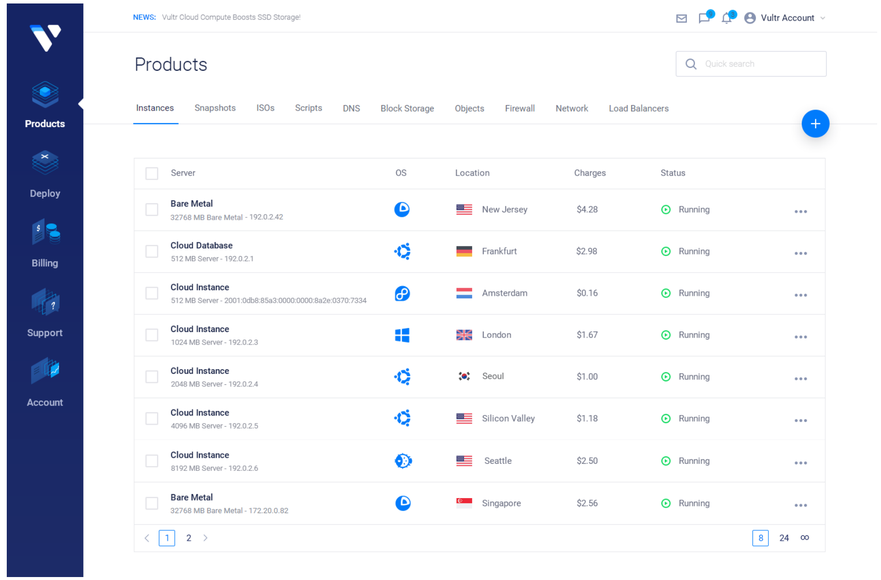
## You can try Linux with hardware and software.

- Virtual Private Servers (VPS): Digital Ocean, Vultr, BandWagon…

DigitalOcean




VULTR



# Class Activity: Get Linux

# Sample Program: toUpper()

# toUpper()

```
#include <stdio.h>
#include <ctype.h>
int main(void)
{
    int c;
    while ((c = getchar()) != EOF) {
        if (islower(c))
            c = toupper(c);
        putchar(c);
    }
    return 0;
}
```

# Write Comment

Bad main() function comment: Describes how the function works

```
Read a character from stdin using getchar.
Depending upon the current DFA state, pass the
character to an appropriate state-handling
function. The value returned by the state-
handling function is the next DFA state. Repeat
until end-of-file. Return 0.
```

Good main() comment: Describes what the function does (from caller's perspective)

```
Read text from stdin. Convert the first character
of each "word" to uppercase, where a word is a
sequence of uppercase or lowercase letters. Write
the result to stdout. Return 0.
```

# Data Types

# C has No Boolean (Logical) Data Type

- Represent logical data using type char or int, or any primitive type! 😱
- But, we have conventions:
  - Statements (if, while, etc.) use 0 ⇒ FALSE, ≠0 ⇒ TRUE
  - Relational operators (<, >, etc.) and logical operators (!, &&, ||) produce the result 0 or 1
  - Would have been nice to have a Boolean type…

# Issues with Lack of Logical Data Type

**Imagine this type of code in Java code and in C. What happens in each case?**

```
…
int i;

…
i = 0;

…
if (i = 5)
    statement1;

…
```

```
…
-10 < i < -1

…
```

# Issues with Lack of Logical Data Type

**Imagine this type of code in Java code and in C. What happens in each case?**

```
…
int i;
…
i = 0;
…
if (i = 5)
    statement1;
…
```

```
…
-10 < i < -1
…
```

- Using integers to represent logical data permits shortcuts; however, it also permits really bad code…

# Issues with Lack of Logical Data Type

```
…
int i;
…
if (i)   /* same as (i != 0) */
    statement1;
else
    statement2;
…
```

- Using integers to represent logical data permits shortcuts; however, it also permits really bad code…

```
i = (1 != 2) + (3 > 4);
```

# Logic Operations

Q: What is **`int i`** set to in the following code?

```
i = (i < (i < 0)) + (i >= (i > 0)) + ((i-i) < (i == i));
```

A. Depends on the initial value of i

B. 0

C. 1

D. 2

E. 3

Q: What is **`int i`** set to in the following code?

```
i = (i < (i < 0)) + (i >= (i > 0)) + ((i-i) < (i == i));
```

A. Depends on the initial value of i

B. 0

C. 1

D. 2

E. 3

D.

If i is negative, this will be 1 + 0 + 1

If i is non-negative, this will be 0 + 1 + 1

# Variable declarations in C89

# Declaring Variables in C

- C requires variable declarations (some languages don’t: awk, bash)
- Declaring variables requires more from the programmer
  - Extra verbiage
  - Thinking ahead about how it will be used
- But it has many benefits
  - Allows compiler to check “spelling”
  - Allows compiler to allocate memory more efficiently
  - Declaring variables’ types produces fewer surprises at runtime

# Declaring Variables in C

- C requires variable declarations (some languages don’t: awk, bash)
- Declaring variables requires more from the programmer
  - Extra verbiage
  - Thinking ahead about how it will be used
- But it has many benefits
  - Allows compiler to check “spelling”
  - Allows compiler to allocate memory more efficiently
  - Declaring variables’ types produces fewer surprises at runtime

# Declaring Variables in C

- C requires variable declarations (some languages don't: awk, bash)
  - Declaration statement specifies type of variable (and other attributes too)

```
int i;
int i, j;
int i = 5;
const int i = 5; /* value of i cannot change */
static int i; /* covered later in course */
extern int i; /* covered later in course */
```

# Declaring Variables in C

- C requires variable declarations (some languages don't: awk, bash)
  - Declaration statement specifies type of variable (and other attributes too)
  - Unlike Java (and later versions of C), declaration statements in C89 must appear before any other kind of statement in any compound statement.

```
{
   int i;

   /* Non-declaration
      statements, even if
      they don't use j */

   int j;
}
```

Illegal in C89

```
{
   int i;
   int j;

   /* Non-declaration
   statements that use
   i and/or j. */
}
```

Legal in C89

# Declaring Variables in C

- C requires variable declarations (some languages don't: awk, bash)
  - Declaration statement specifies type of variable (and other attributes too)
  - Unlike Java (and later versions of C), declaration statements in C89 must appear before any other kind of statement in any compound statement.

```
{
   int i;

   /* Non-declaration
      statements, even if
      they don't use j */

   int j;
}
```

Illegal in C89

```
{
   int i;
   int j;

   /* Non-declaration
   statements that use
   i and/or j. */
}
```

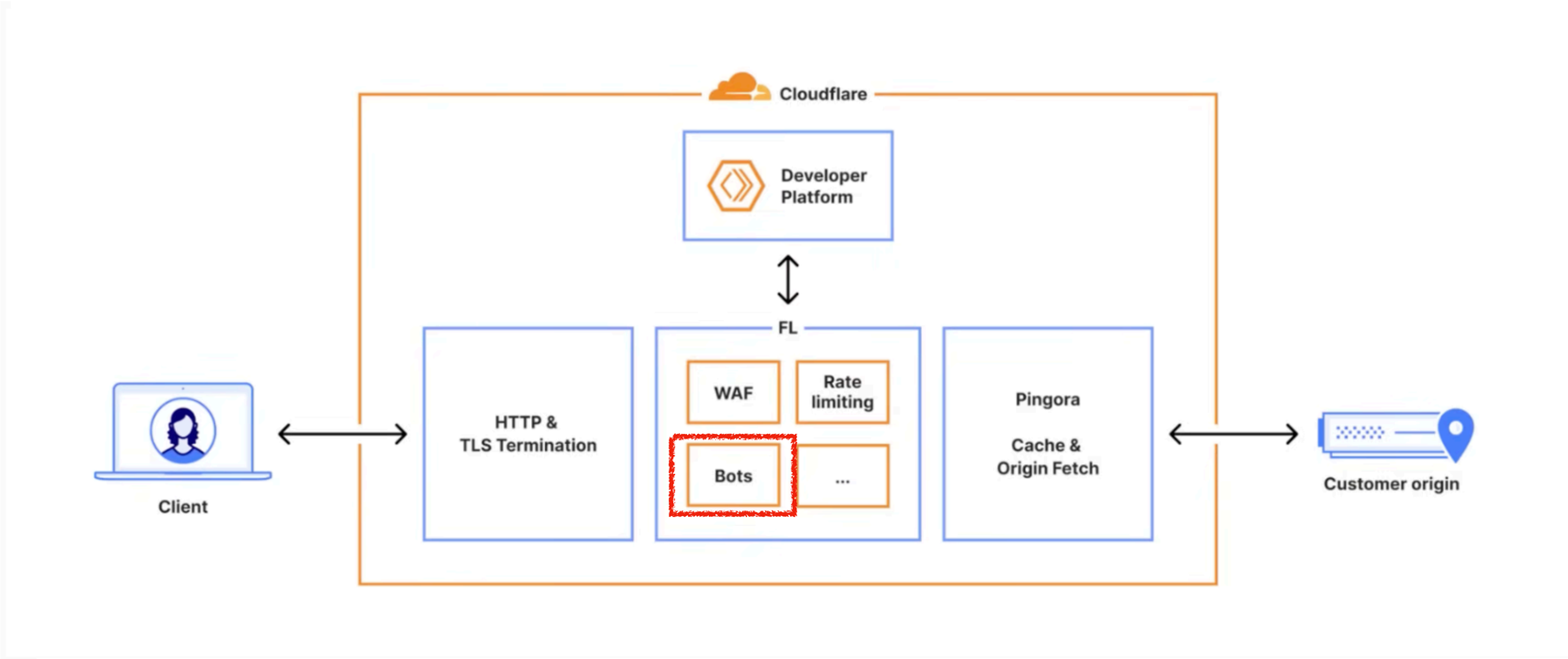
Legal in C89

**Note: this doesn't mean that all declarations must be at the top of a function exclusively!**

# Case Study:
# Cloudflare Downtime(11/18/’25)

# Services by Cloudflare

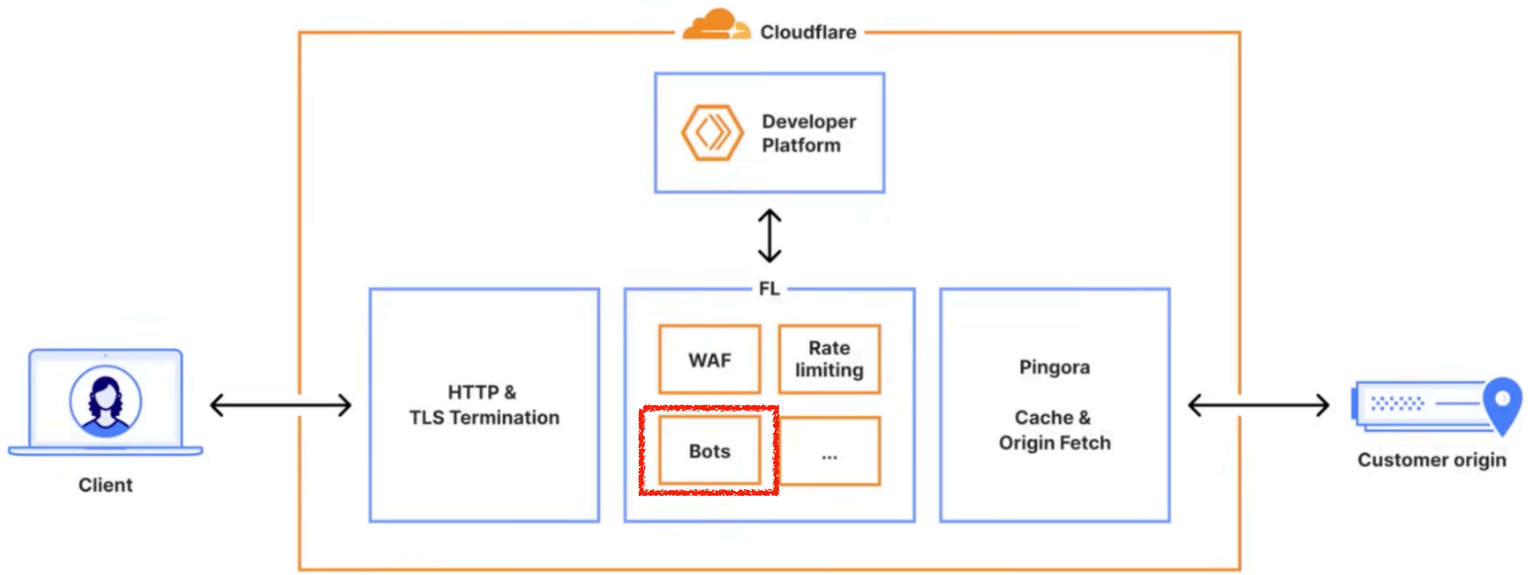
## Content Delivery Network toolkits for web service providers.

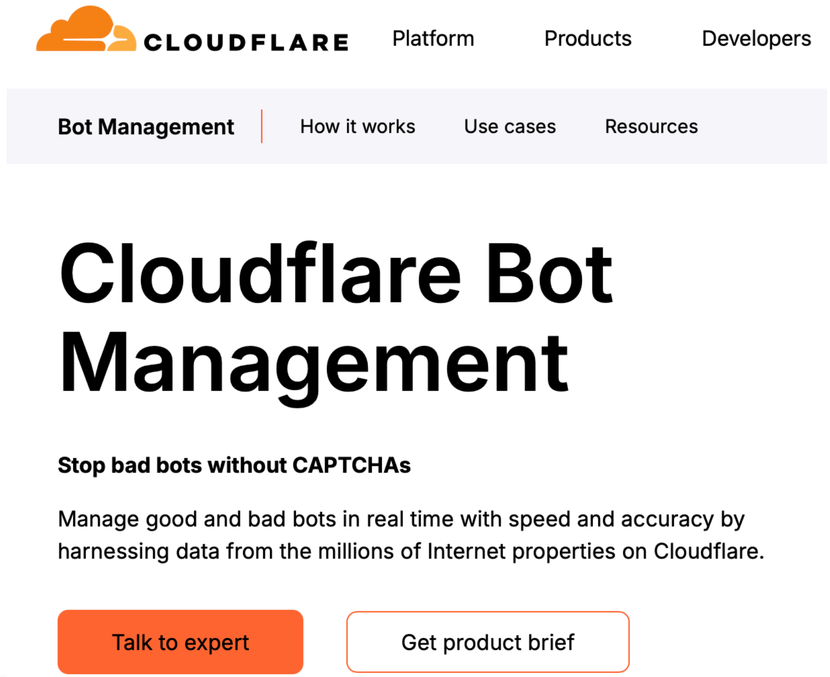



- Debug, DDoS, Speed Up, …

# Services by Cloudflare

## Content Delivery Network toolkits for web service providers.

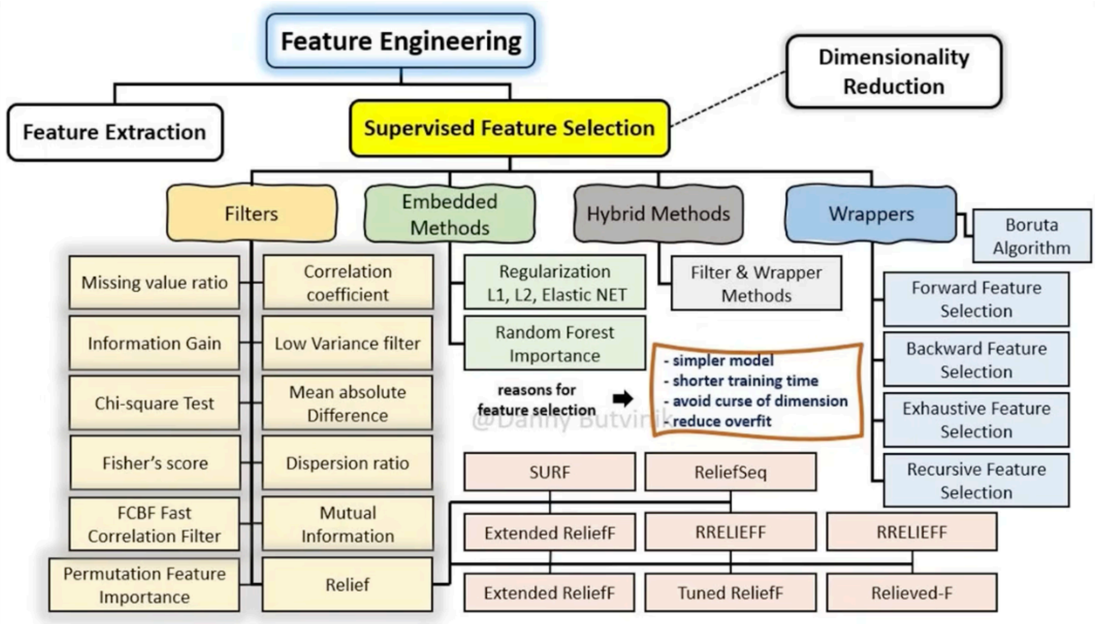



- Debug, DDoS, Speed Up, …
- Bot Management: Detect suspicious HTTP requests with Machine Learning.

# Services by Cloudflare

## Content Delivery Network toolkits for web service providers.



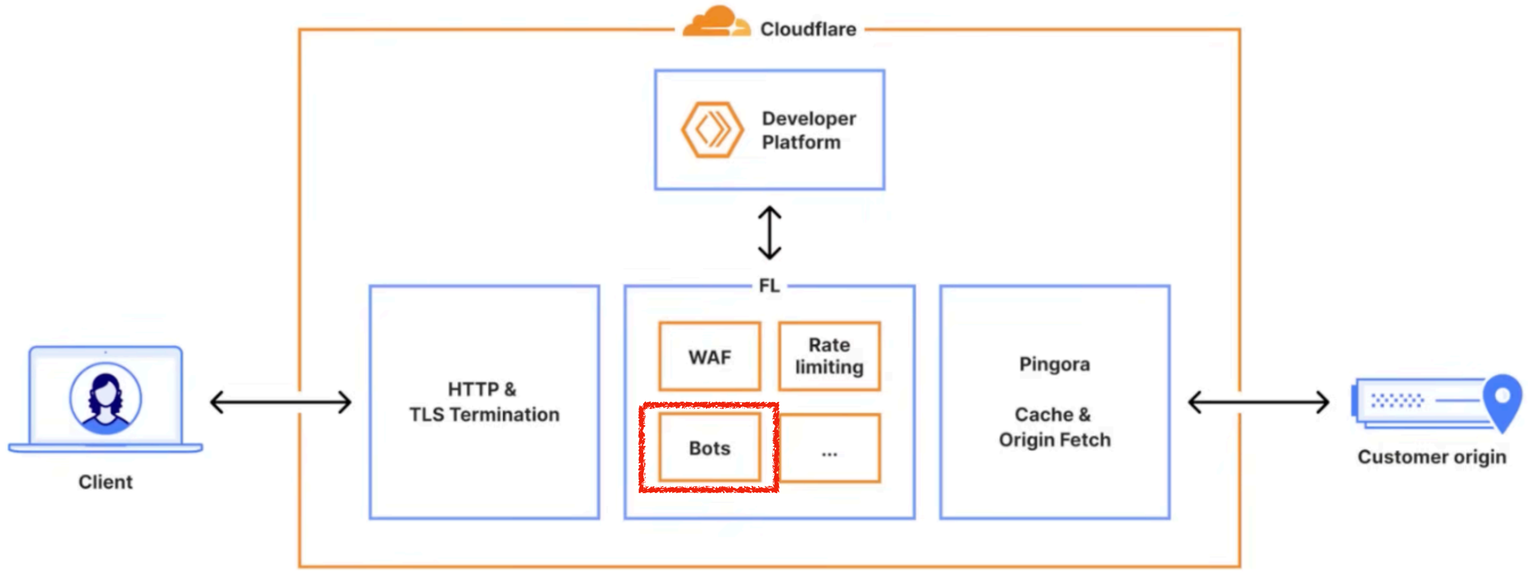


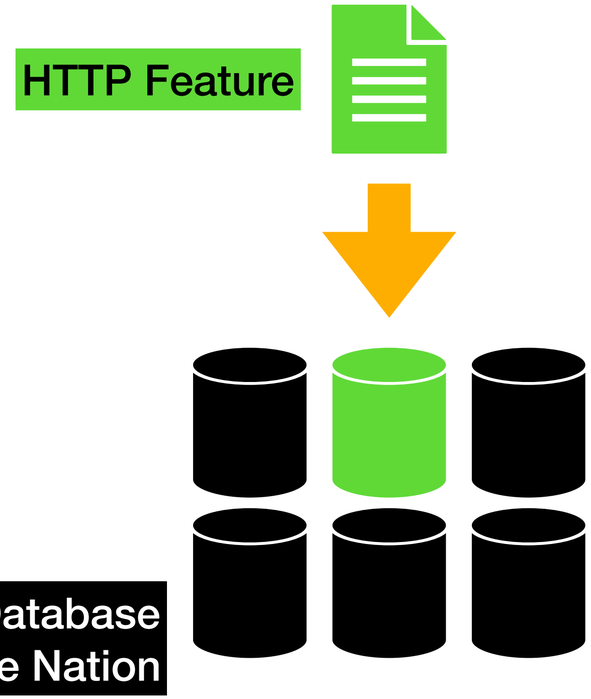


- Debug, DDoS, Speed Up, …
- Bot Management: Detect suspicious HTTP requests with Machine Learning.

# Services by Cloudflare

**Content Delivery Network toolkits for web service providers.**




- Debug, DDoS, Speed Up, …
- Bot Management: Detect suspicious HTTP requests with Machine Learning.

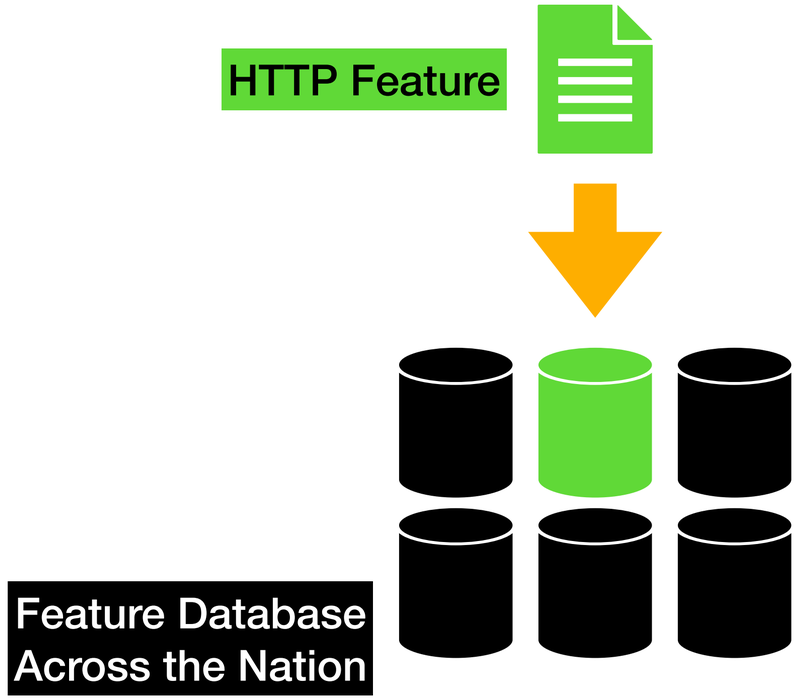
# Traceback

## Before 11:05

```
SELECT name, type
FROM system.columns
WHERE table = 'http_requests_features'
order by name;
```

```
SELECT feature1, feature2, ...
FROM r0.http_requests_features;
```



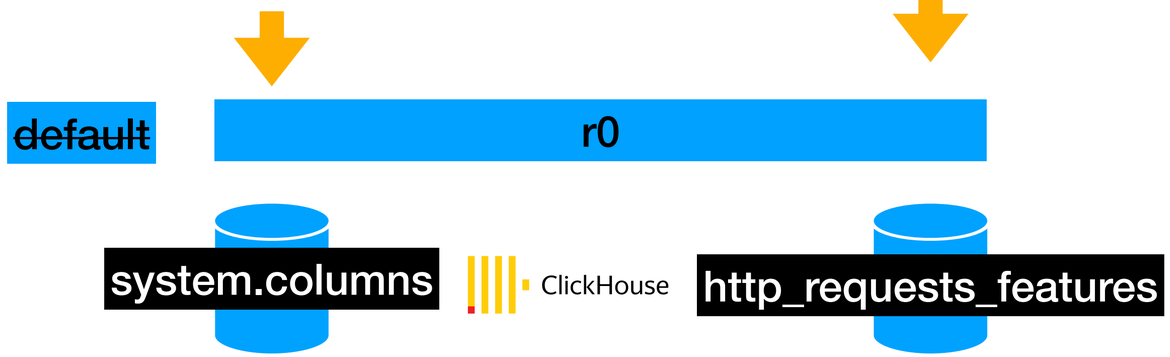

- Debug, DDoS, Speed Up, …
- Bot Management: Detect suspicious HTTP requests with Machine Learning.

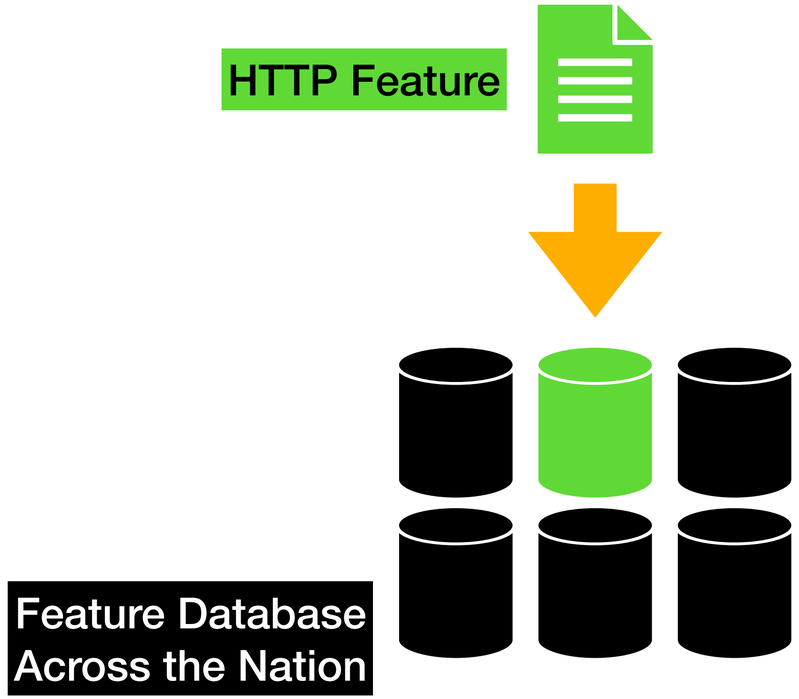
# Traceback

## Update at 11:05

```
SELECT name, type
FROM system.columns
WHERE table = 'http_requests_features'
order by name;
```

```
SELECT feature1, feature2, ...
FROM r0.http_requests_features;
```



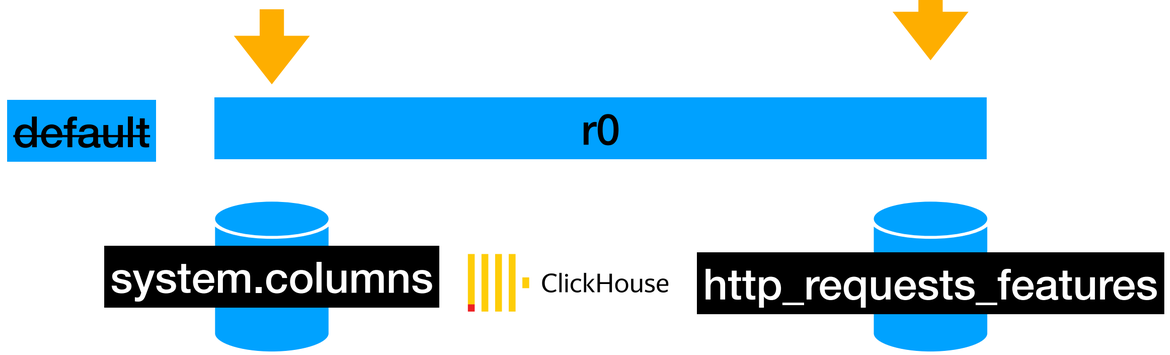

- Debug, DDoS, Speed Up, …
- Bot Management: Detect suspicious HTTP requests with Machine Learning.

# Traceback

## Update at 11:05

```
SELECT name, type
FROM system.columns
WHERE table = 'http_requests_features'
order by name;
```

```
SELECT feature1, feature2, ...
FROM r0.http_requests_features;
```




**BEFORE**

| database | table | name | type | ... |
|---|---|---|---|---|
| default | http_requests_features | feature1 | int | ... |
| default | http_requests_features | feature2 | int | ... |
| default | http_requests_features | feature3 | int | ... |

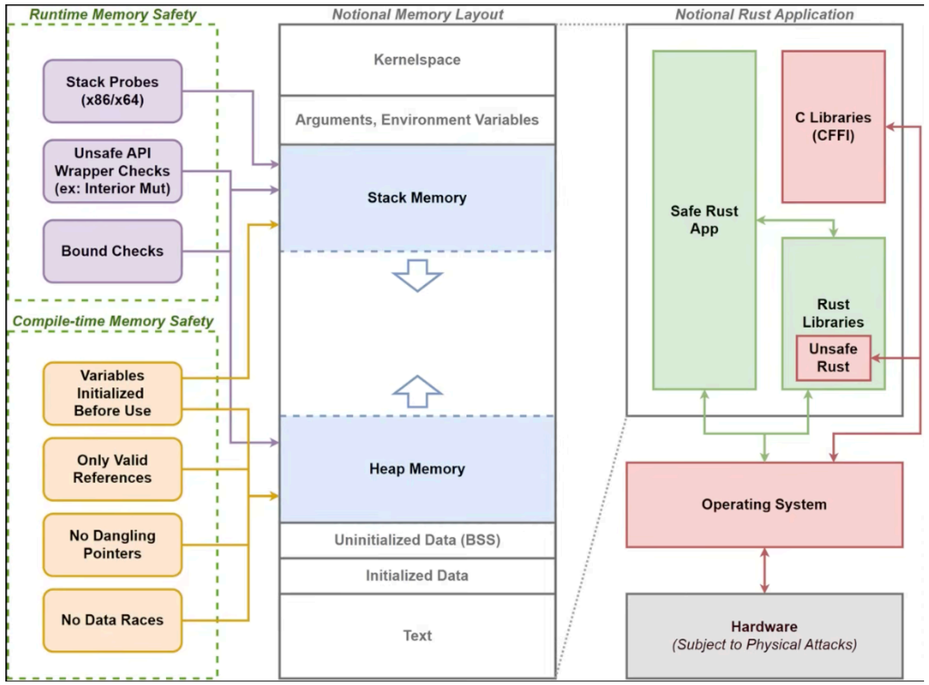
**AFTER**

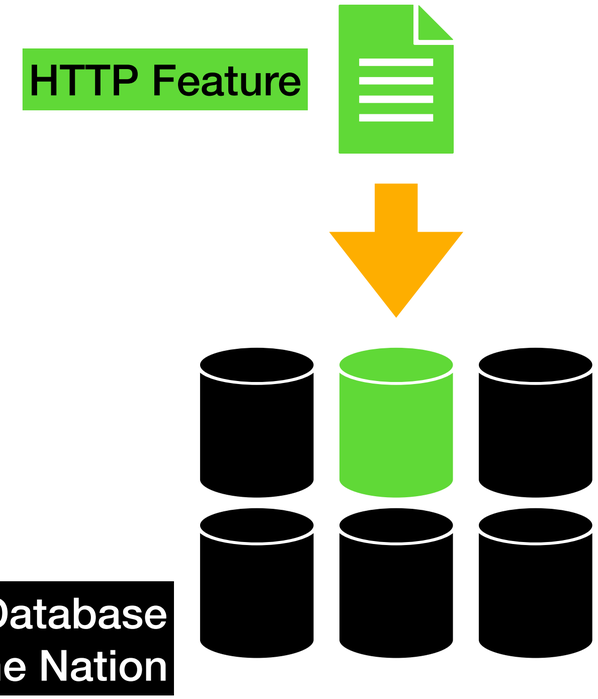
| database | table | name | type | ... |
|---|---|---|---|---|
| default | http_requests_features | feature1 | int | ... |
| default | http_requests_features | feature2 | int | ... |
| default | http_requests_features | feature3 | int | ... |
| r0 | http_requests_features | feature1 | int | ... |
| r0 | http_requests_features | feature2 | int | ... |
| r0 | http_requests_features | feature3 | int | ... |

- Privilege setting with ClickHouse =
- Bot Management: Taking duplicated features everytime.

# Traceback

## Update at 11:05



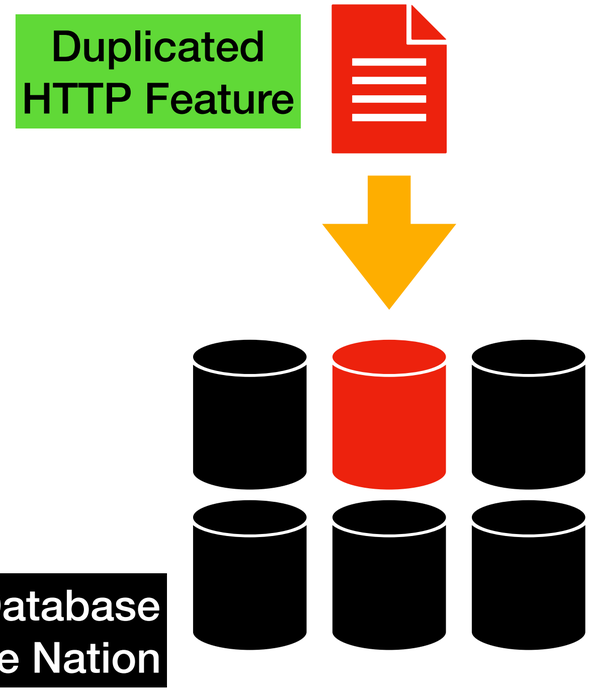




- Pre-allocation Memory
- Limit features to less than 200 -> Magic Number

# Traceback

## Update at 11:18

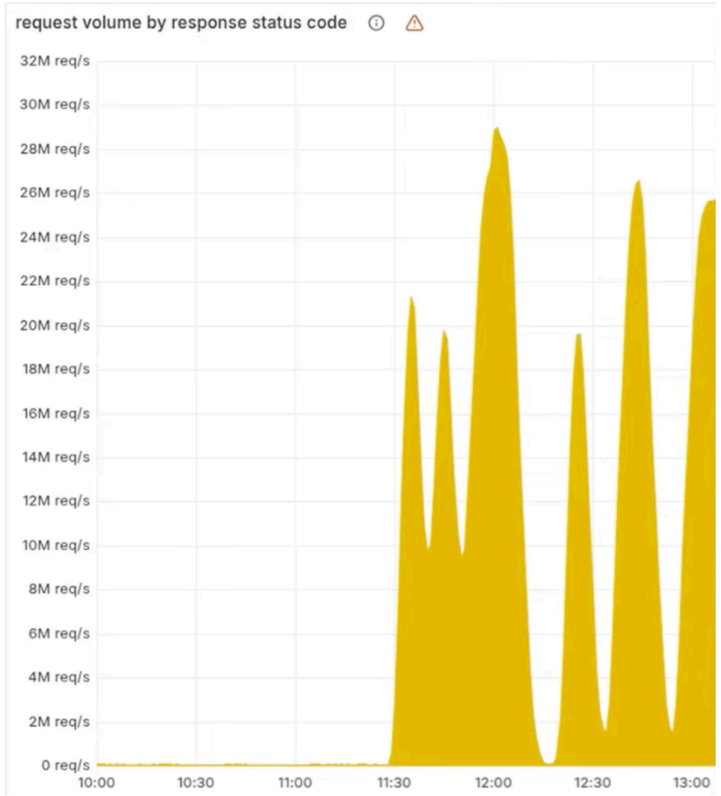
```
71      /// Fetch edge features based on `input` struct into [`Features`] buffer.
72      pub fn fetch_features(
73          &mut self,
74          input: &dyn BotsInput,
75          features: &mut Features,
76      ) -> Result<(), (ErrorFlags, i32)> {
77          // update features checksum (lower 32 bits) and copy edge feature names
78          features.checksum &= 0xFFFF_FFFF_0000_0000;
79          features.checksum |= u64::from(self.config.checksum);
80          let (feature_values, _) = features
81              .append_with_names(&self.config.feature_names)
82              .unwrap();
```

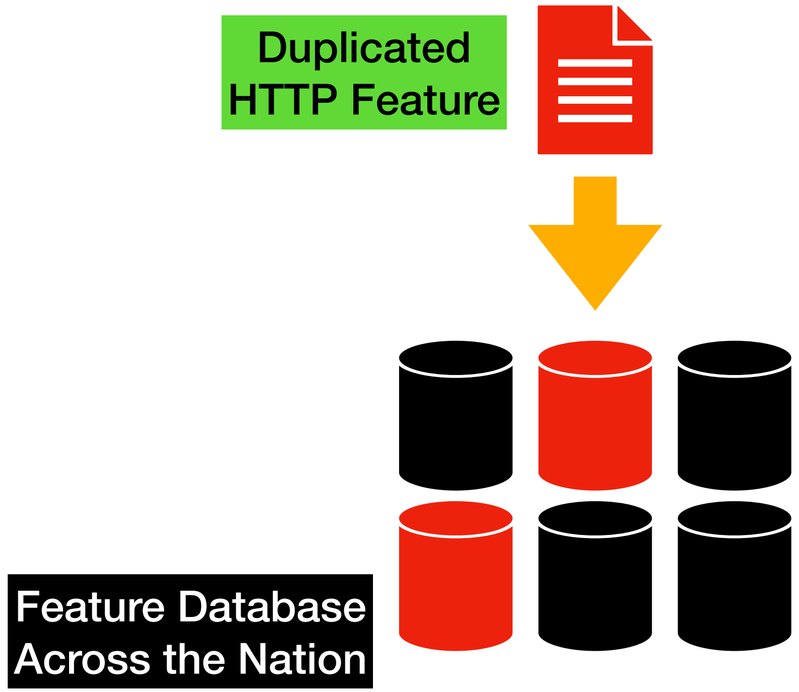



- Calling .unwrap()
- Pre-allocated Memory Breached -> No HTTP Request Handling

# Traceback

**11:32**



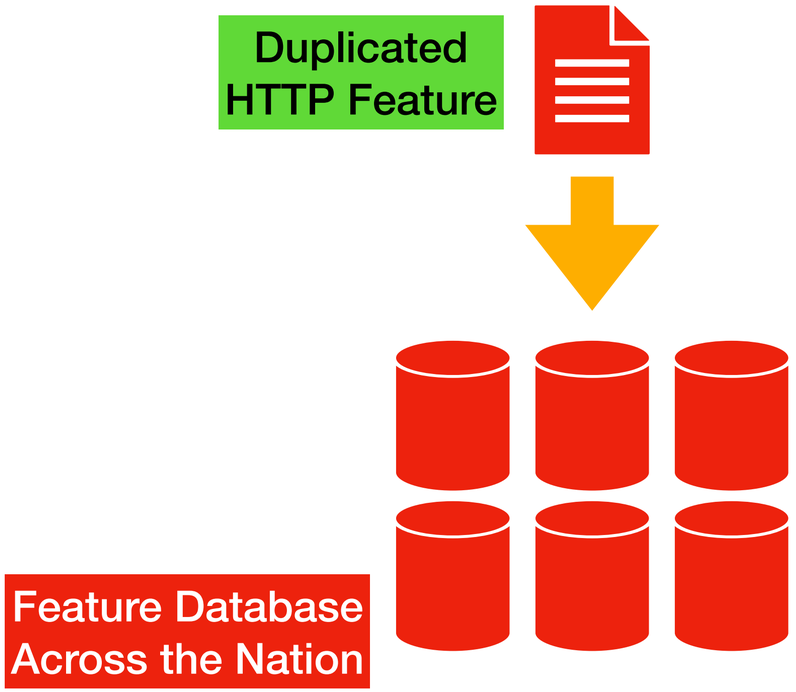




- Calling .unwrap() -> Panic & Halt
- Ill-formed feature across databases
- Not all servers stopped as some database is still clean

# Traceback

**13:30**



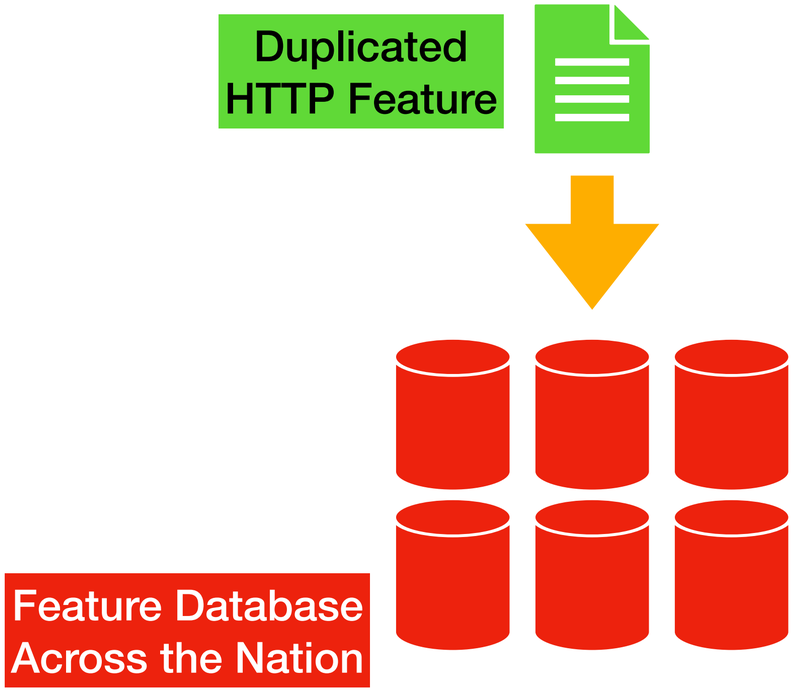




- Calling .unwrap() -> Panic & Halt
- Ill-formed feature across databases
- All feature databases are corrupted & panicking servers -> no services

# Traceback

## 14:30

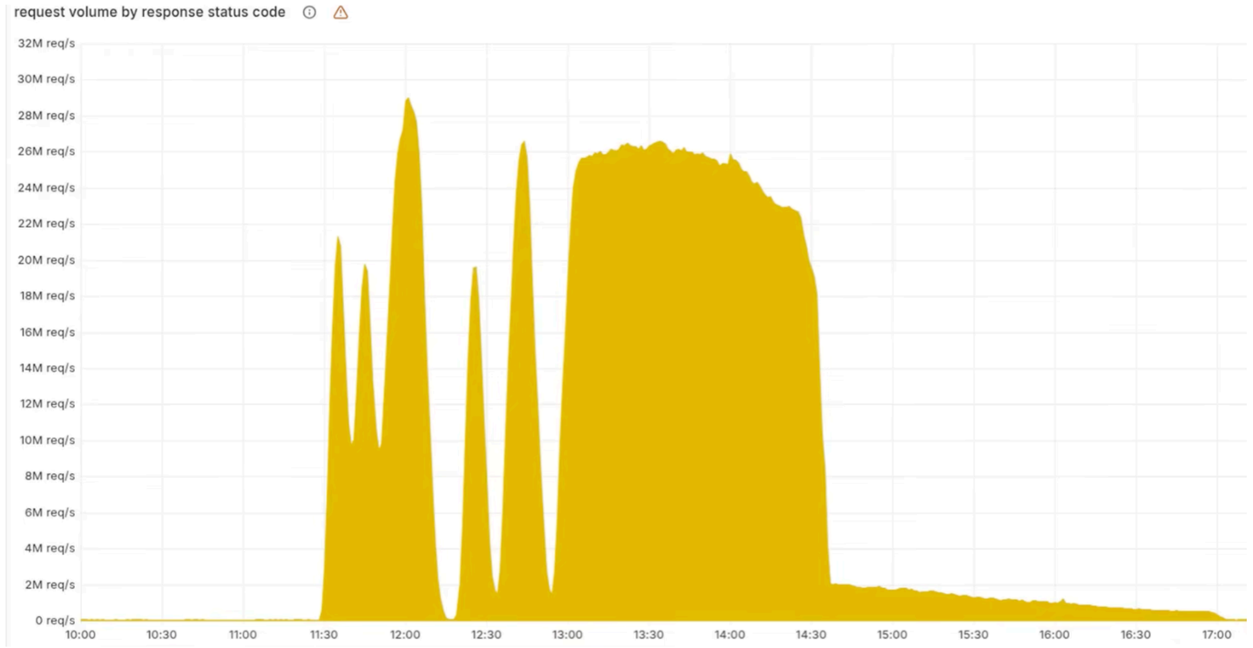


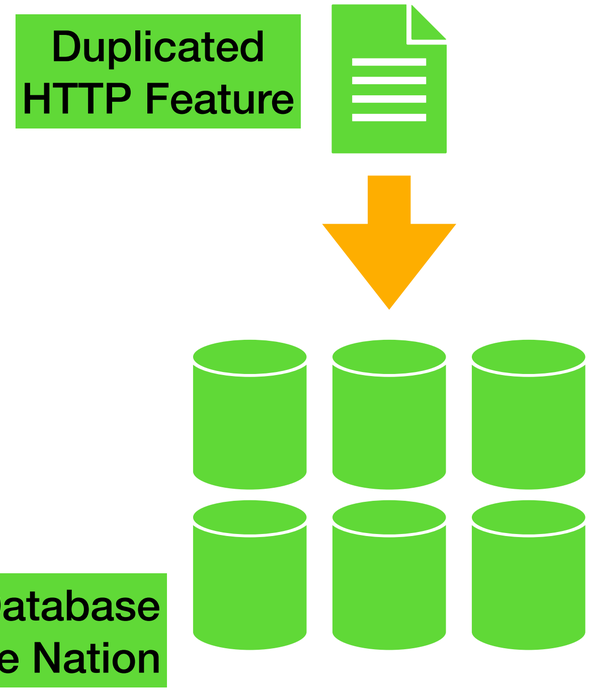




- Realize this is not DDoS attack.
- Rewrite Privilege Protocols
- Create Clean HTTP Feature File & Stop Automatic Feature Broadcasting

# Traceback

## 14:36




- HTTP feature pushed at 14:30.
- 6 mins later, all feature databases restored.

# Things to Consider…

**What are the elements to this disaster? What can we do to prevent?**

# Things to Consider…

**What are the elements to this disaster? What can we do to prevent?**

- Lack of Understanding to Side Effects
- Mis-assumptions to Database Behavior
- Magic Numbers

**Just as Program = Algorithm + Data Structure, or Program = Data, bugs are always data related. Consider in the program's side (control flow) AND data your program is processing, gives you complete tool to locate the problems and save the world.**

# Concept: Indirection

# What is Indirection?

- Referencing an object through an intermediary, such as a pointer, handle, or a layer of abstraction, rather than directly
  - Provides decoupling
  - Which helps manage complexity and provide flexibility in programming and system design
  - Critical in many areas of CS: databases, Internet (e.g. DNS), virtual machines, design patterns, …

"All problems in computer science can be solved by another level of indirection”

"All problems in computer science can be solved by another level of indirection”*

*But can get complicated if not managed well, and can reduce performance

# String

# String at First Look

```
char string[10] =

{'H','e','l','l','o','\0'};

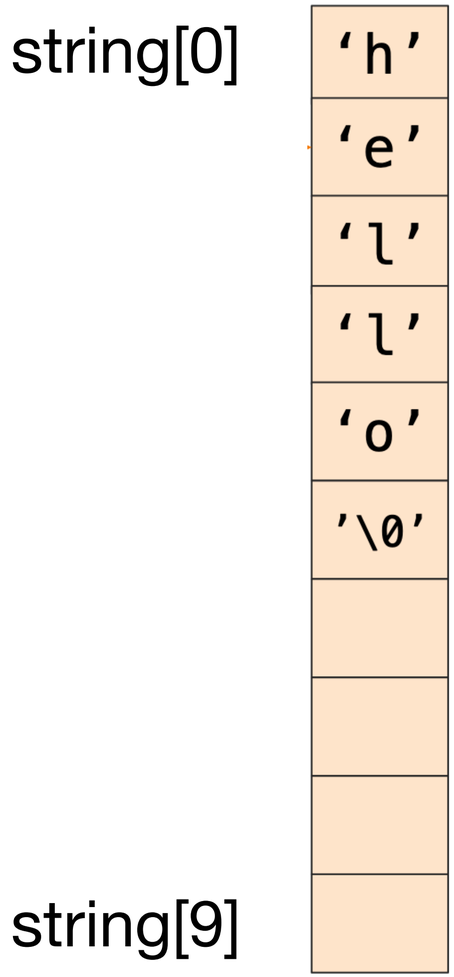
(or, equivalently*)

char string[10] = "Hello";

char *pc = string+1;

printf("Y%sw ", &string[1]);

printf("J%s!\n", pc);
```



# String at First Look

**How could we manipulate the data inside of a piece of memory,**
**If we see the string as an indirection?**

```
char string[10] =

{'H','e','l','l','o','\0'};

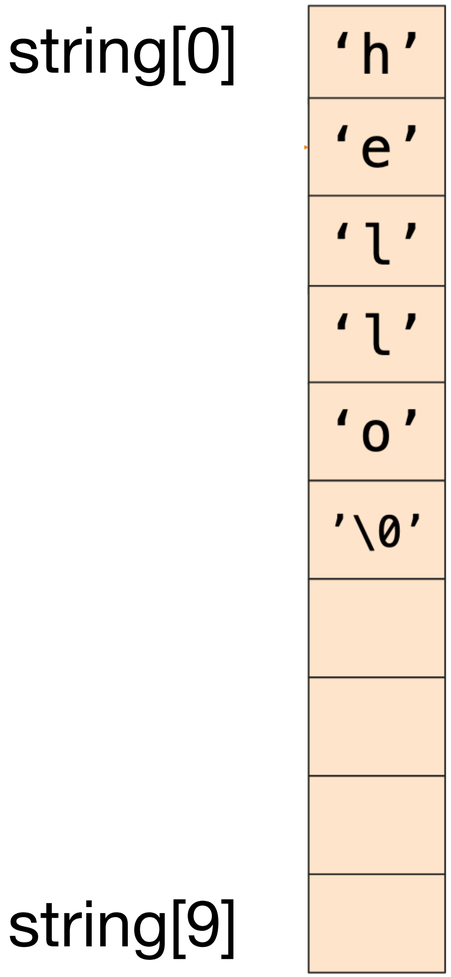
(or, equivalently*)

char string[10] = "Hello";

char *pc = string+1;

printf("Y%sw ", &string[1]);

printf("J%s!\n", pc);
```



# String at First Look

**How could we manipulate the data inside of a piece of memory,**
**If we see the string as an indirection?**

```
char string[10] =

{'H','e','l','l','o','\0'};

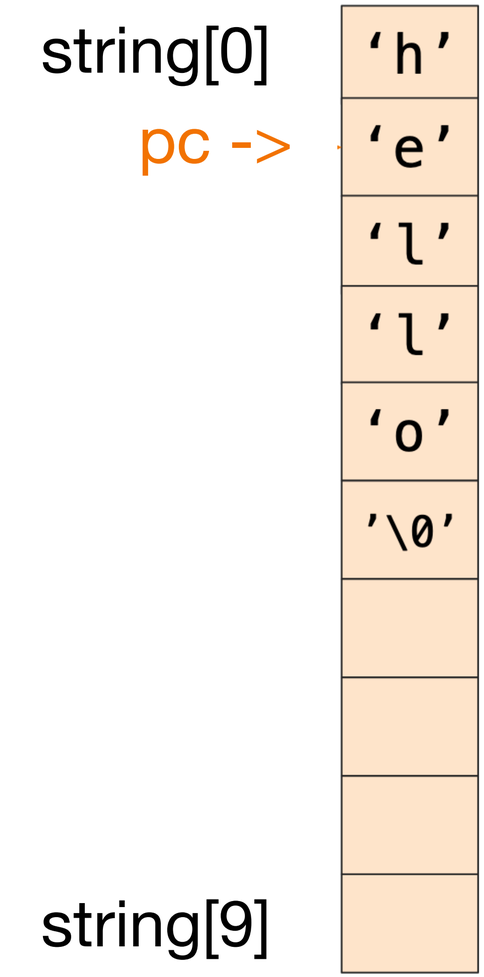
(or, equivalently*)

char string[10] = "Hello";

char *pc = string+1;

printf("Y%sw ", &string[1]);

printf("J%s!\n", pc);
```



# Pointers

# What is a Pointer?

- A pointer is a variable
- Its value is the location (addr) of another variable
- “Dereference” (follow) the pointer to read/write the value at that location

# What is a Pointer?

- A pointer is a variable
- Its value is the location (addr) of another variable
- “Dereference” (follow) the pointer to read/write the value at that location



# What do I need to set a Pointer?

**Recall last time we had to move to another classroom upstairs…**

- Like a variable, a pointer needs a name.
- The value is the location (address) of another variable.
- As we are dealing with locations, we need the size of the other variable.

myCSCI260Sticker






```
classroom * myCSCI260Sticker = THE LOCATION OF BC27
```

# What do I need to set a Pointer?

**Recall last time we had to move to another classroom upstairs…**

- Like a variable, a pointer needs a name.
- The value is the location (address) of another variable.
- As we are dealing with locations, we need the size of the other variable.

myCSCI260Sticker






```
classroom * myCSCI260Sticker = &BC27
```

# What is a Pointer?

- A pointer is a variable
- Its value is the location (addr) of another variable
- “Dereference” (follow) the pointer to read/write the value at that location

**Why is that a good idea?**

- Copying pointers is much faster than large structures
- x=y is a one-time copy: if y changes, x doesn’t “update”
- Parameters to functions are copied; but handy to be able to modify value
- Often need a handle to access dynamically allocated memory

# Pointers in C

**Use a pointer is as easy as pulling out a sticker.**

- First, pull out a sticker.
- Then, write what the sticker is about.
- Next, give the sticker a name.
- Lastly, write where this sticker is “pointing at” / “refer to” the destination.
- It is okay for you don’t know the exact destination, use ‘&’ before the name of the variable, and the machine will look it up for you.

# Pointers in C

```
int a = 150;

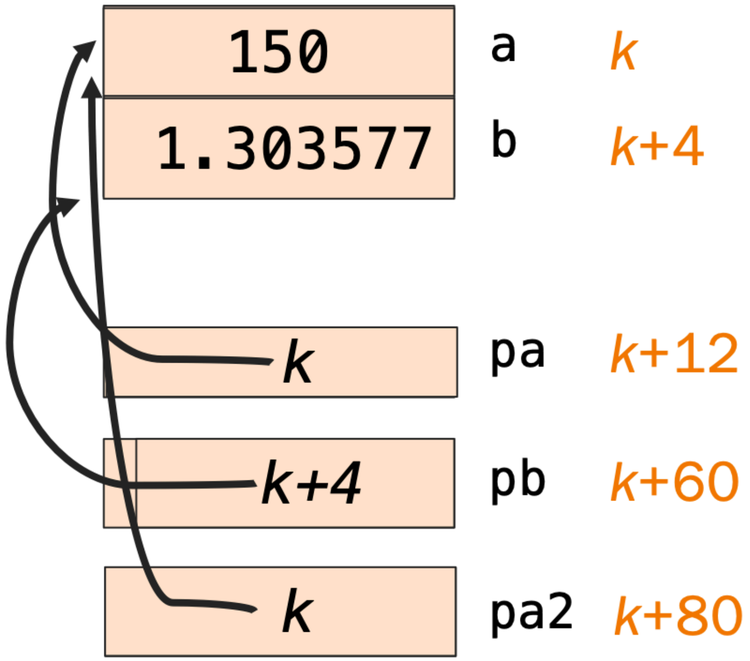
double b = 1.303577;

int *pa = NULL;

double *pb = &b;

pa = &a;

*pa = (int) *pb;
```



# Pointers in C

```
int a = 150;

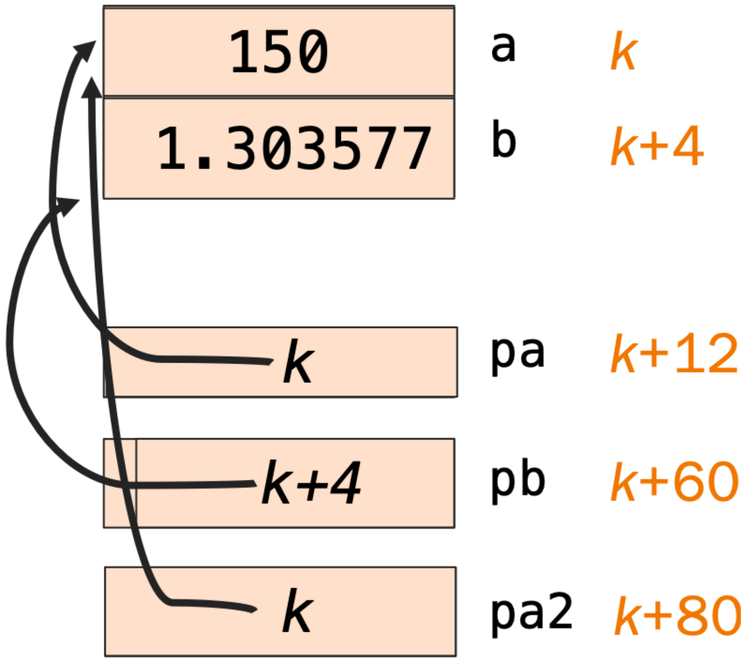
double b = 1.303577;

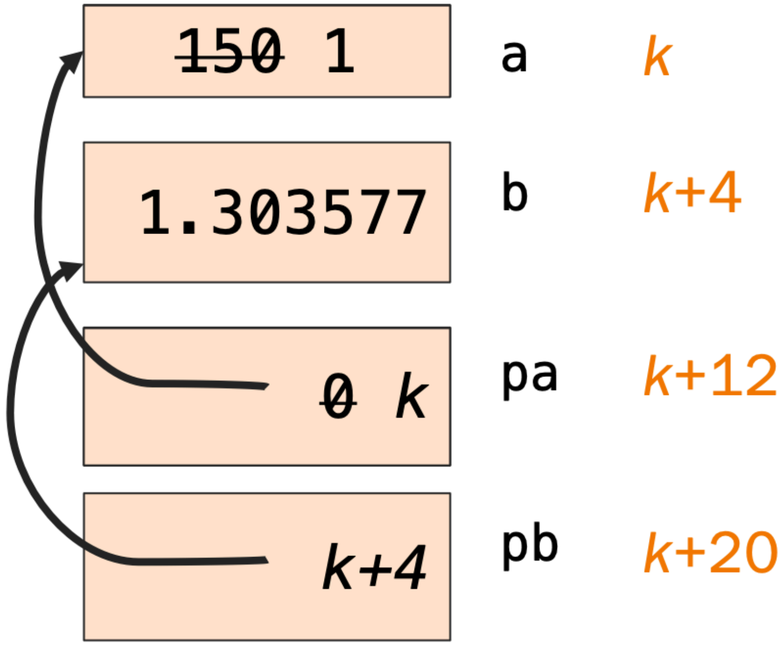
int *pa = NULL;

double *pb = &b;

pa = &a;

*pa = (int) *pb;
```



# Pointers in C

**Pointer types are target dependent**

- Example: “int *pi;” – declares pi to be a pointer to an int
- We’ll see “generic” pointers later

**Pointer values are memory addresses**

- … so size is architecture-dependent – 8 bytes on ARMv8
- NULL is macro in stddef.h for special pointer guaranteed

not to point to any variable

# Pointers in C

**Pointer-specific operators**

- Address-of operator (&a) – a pointer to variable “a”
- Dereference operator (*a) – value that ptr “a” points to

**Other pointer operators are available**

- Assignment operator: =
- Relational operators: ==, !=, >, <=, etc.
- Arithmetic operators: +, −, ++, −=, !, etc.

# Pointers in C

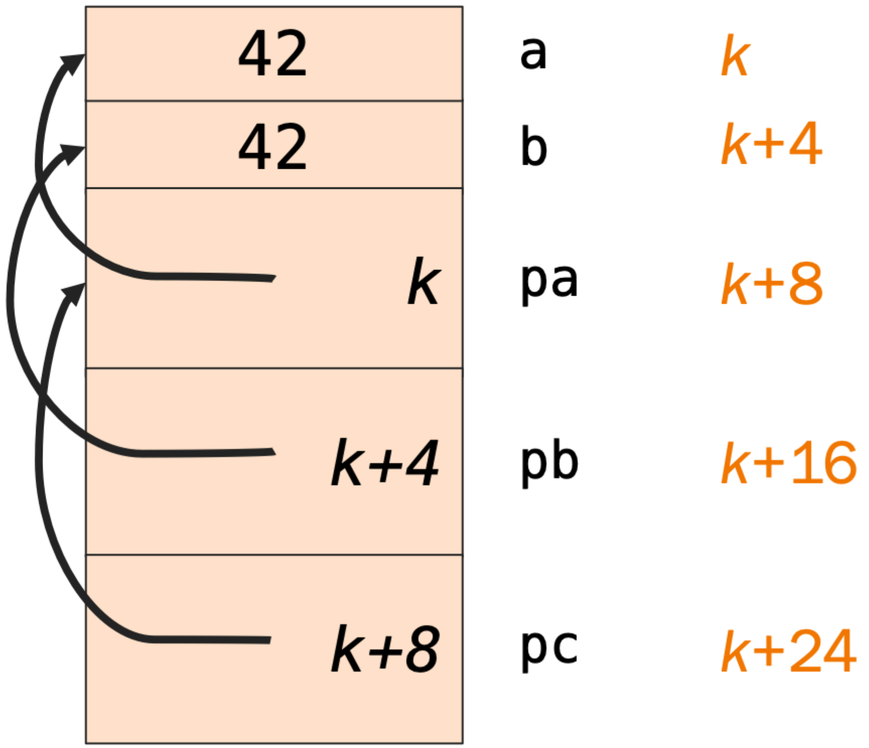
```
int a = 42;
int b = 42;
int *pa = &a;
int *pb = &b;
int **pc = &pa;
printf("%d %d\n",
               pa ==  pb,
              *pa == *pb);
printf("%d %d %d %d %d\n",
               pc == &pa,
               pc == &pb,
              *pc ==  pa,
              *pc ==  pb,
             **pc == *pb);
```

What if this were:
`int *pa;`
`*pa = &a;`
Is it the same?

```
0 1
```

```
1 0 1 0 1
```

```
<- same as *pa == *pb
```



# Pointer Declaration Gotcha

Pointer declarations can be written as follows: `int* pi;`

This is equivalent to: `int *pi;`

but the former seemingly emphasizes that the type of pi is ("int pointer”)

Even though the first syntax may seem more natural, and you are welcome to use it, the * is attached to the variable name (pi), not to the type.

# Pointer Declaration Gotcha

>>> Beware <<<

This declaration: `int* p1, p2;`

really means: `int *p1; int p2;`

To declare both p1 and p2 as pointers, i.e.: `int* p1; int* p2;`

in one statement, must "star" both variables: `int *p1, *p2;`

# Array and Pointer

# Recall: Array in Java

Always dynamically allocated

- Even when the values are known at compile time (e.g., initializer lists)
- Objects on the heap.

Access via reference variable (handle)

- Dereference, assign/copy. Can’t see value.

# Recall: Array in Java

Always dynamically allocated

- Even when the values are known at compile time (e.g., initializer lists)
- Objects on the heap.

Access via reference variable (handle)

- Dereference, assign/copy. Can't see value.

```
public static void arrays() {

int[] arr1 = {1, 2, 3};

int[] arr2 = new int[3];

int[] arr3 = arr1;

for (int c = 0;

c < arr2.length; c++)

arr2[c] = 3 * arr1[c];

}
```

# Recall: Array in Java

Always dynamically allocated

- Even when the values are known at compile time (e.g., initializer lists)
- Objects on the heap.

Access via reference variable (handle)

- Dereference, assign/copy. Can't see value.

```
public static void arrays() {
int[] arr1 = {1, 2, 3};
int[] arr2 = new int[3];
int[] arr3 = arr1;
for (int c = 0;
c < arr2.length; c++)
arr2[c] = 3 * arr1[c];
}
```

# Recall: Array in Java

Always dynamically allocated

- Even when the values are known at compile time (e.g., initializer lists)
- Objects on the heap.

Access via reference variable (handle)

- Dereference, assign/copy. Can't see value.

```
public static void arrays() {

int[] arr1 = {1, 2, 3};

int[] arr2 = new int[3];

int[] arr3 = arr1;

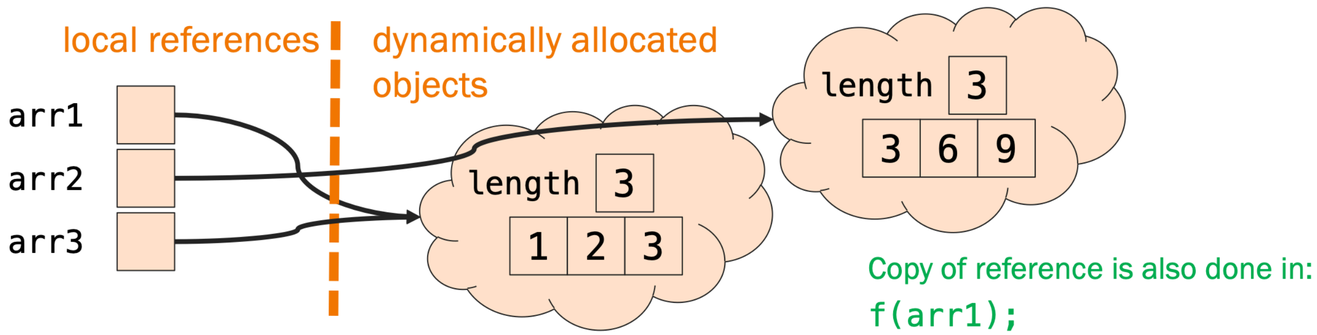
for (int c = 0;

c < arr2.length; c++)

arr2[c] = 3 * arr1[c];

}
```

# Recall: Array in Java




Access via reference variable (handle)

- Dereference, assign/copy. Can't see value.

```
public static void arrays() {

int[] arr1 = {1, 2, 3};

int[] arr2 = new int[3];

int[] arr3 = arr1;

for (int c = 0;

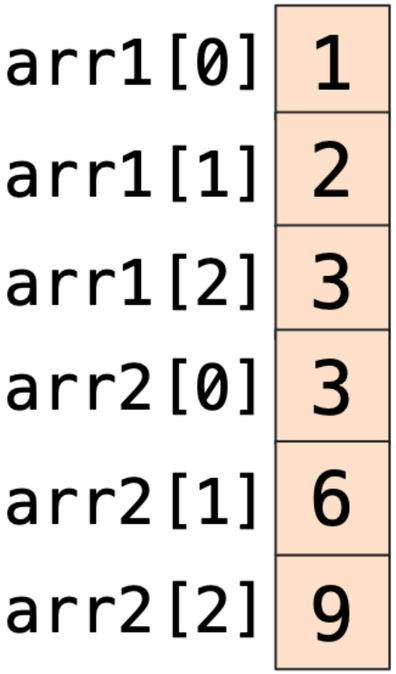
c < arr2.length; c++)

arr2[c] = 3 * arr1[c];

}
```

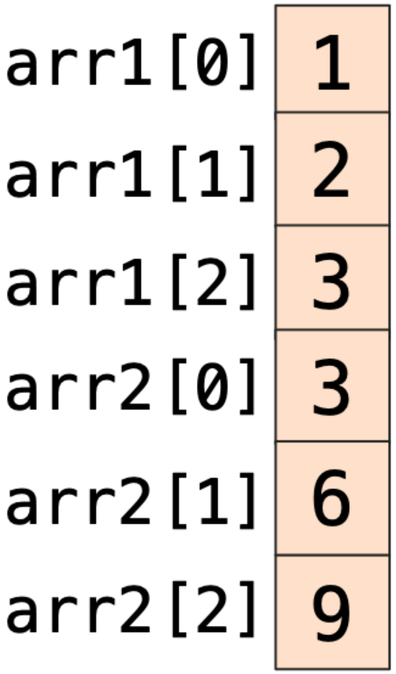
# C Arrays

- Can be statically allocated, e.g., as local variables
- Length of the array must be known at compile time.
- Can also be dynamically allocated.



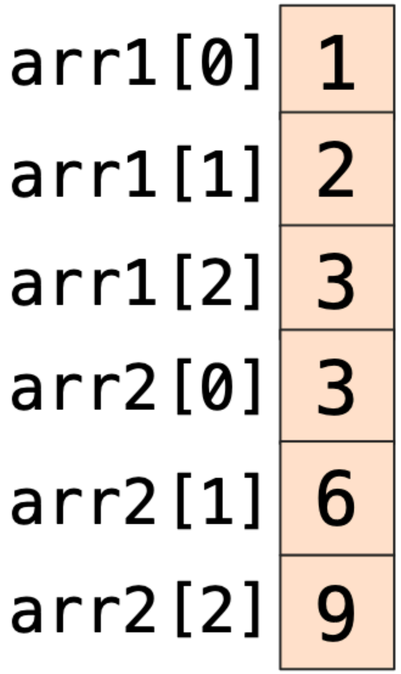
# C Arrays

```
void arrays() {
int c;
int arr1[] = {1, 2, 3};
int arr2[3];
~~int[] arr3 = arr1;~~
int arr2len =
sizeof(arr2)/sizeof(int);
for (c = 0; c < arr2len; c++)
arr2[c] = 3 * arr1[c];
}
```



# C Arrays

```
void arrays() {
int c;
int arr1[] = {1, 2, 3};
int arr2[3];
~~int[] arr3 = arr1;~~
int arr2len =
sizeof(arr2)/sizeof(int);
for (c = 0; c < arr2len; c++)
arr2[c] = 3 * arr1[c];
}
```




# Java Arrays

```
public static void arrays() {
int[] arr1 = {1, 2, 3};
int[] arr2 = new int[3];
for(int c = 0;
c < arr2.length; c++)
arr2[c] = 3 * arr1[c];
int[] arr3 = arr1;
}
```